Innovación en Formación Profesional

Proyecto de big data e inteligencia artificial

Fase 2

CURSO DE ESPECIALIZACIÓN EN

INTELIGENCIA ARTIFICIAL Y BIG DATA

MODALIDAD ONLINE

Daniel Chacón Bautista

Javier Quiñonero González

David Bartol Cortés

Álvaro Benito Diaz

16/04/2026

https://github.com/danielchb01/GesturaAI

Producto Funcional en Local + Código **3**

1. Estado actual del proyecto 3
2. Guía de ejecución local 3
3. Evidencias funcionales 4
4. Arquitectura real implementada 8
5. Índice de código (estructura del proyecto) 10
6. Código completo del proyecto 11
7. Breve explicación por módulo 23
8. Limitaciones actuales 24

Producto Funcional en Local + Código

1. Estado actual del proyecto

- Funcionalidades completas
  - **Pipeline de Visión:** Captura de vídeo optimizado con manejo de errores y soporte para diferentes backends de cámara (DSHOW, MSMF).
  - **Motor de Juego:** Lógica de "Piedra, Papel, Tijera" desacoplada de la interfaz, permitiendo cambios de reglas sin afectar el resto.
  - **UI Dinámica:** Interfaz de usuario con marcador, estados de juego y feedback visual de landmarks.
  - **Arquitectura Modular:** Separación clara de responsabilidades en carpetas (src, models, ml, data).
  - **Integración ML:** Carga automática de modelos .pkl y codificadores de etiquetas.
- Funcionan parcialmente
  - **Detección de gestos:** El sistema está listo, pero la precisión real depende del ángulo y luz de la mano.
  - **HandProcessor Híbrido:** El código ya detecta automáticamente si debe enviar 42 o 63 parámetros, pero actualmente solo se usa en modo 2D.
- Pendientes para Fase 3
  - **Persistencia con SQLite:** Implementación de una base de datos local para registrar el histórico de partidas (gestos usados, resultados, etc...)
  - **Evolución a 3D (63 features):** Re-entrenamiento con dataset para mejorar la presión en gestos inclinados
  - **Efectos de Sonido (SFX):** Integración para el "3,2,1" y para el resultado de la partida
  - **Menú Principal:** Pantalla de inicio antes de entrar directamente al flujo de la cámara.

2. Guía de ejecución local

- Clonar el repositorio
- Crear y activar entorno virtual
- Instalar dependencias del fichero requirements.txt
- Ejecutar la aplicación

3. Evidencias funcionales

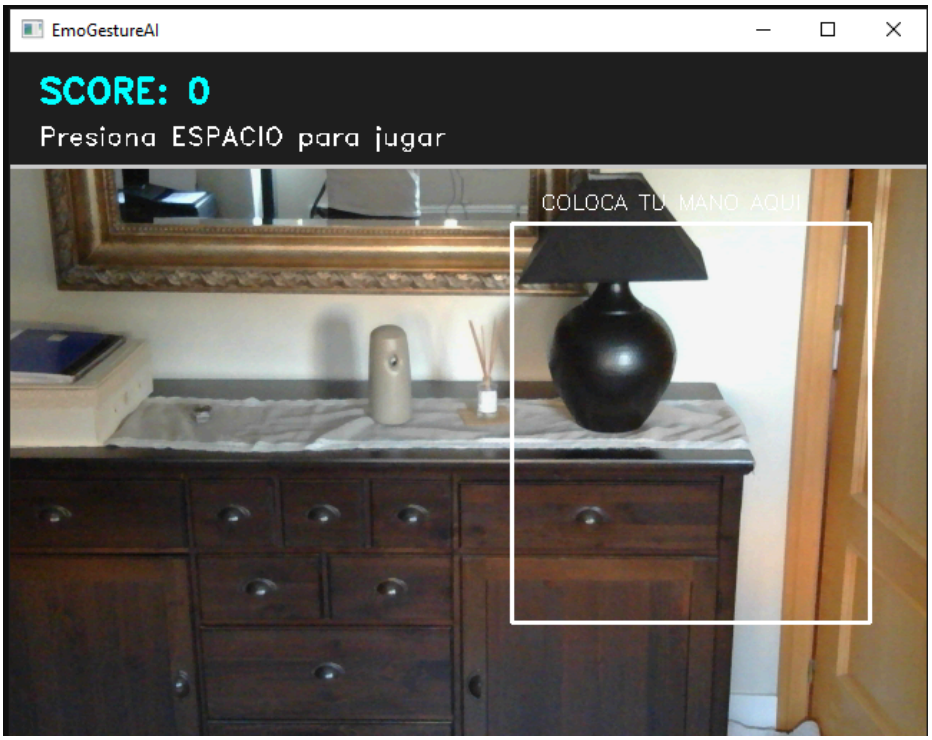
- Arranca la aplicación, pero la cámara no está conectada

```
(env) D:\Dani\Documentos\EmoGestureAI\src>python main.py
[ WARN:0@2.571] global cap.cpp:480 cv::VideoCapture::open VIDEOIO(DSHOW): backend is generally available but can't be used to capture by index
ERROR: No se detecta la cámara. Revisa la conexión.
Aplicación iniciada. Presiona 'q' para salir.
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

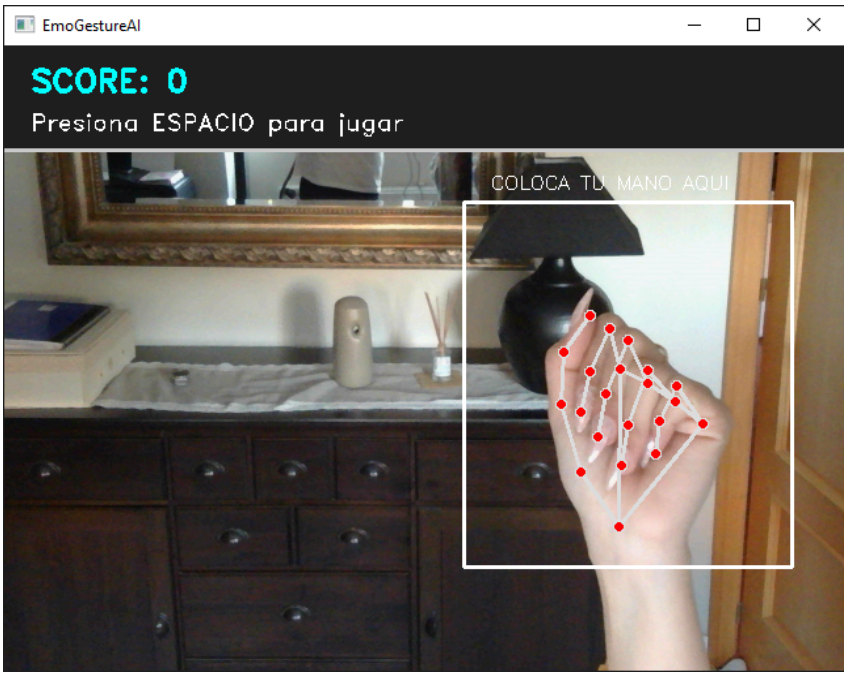
- Arranca aplicación

```
(env) D:\Dani\Documentos\EmoGestureAI\src>python main.py
Aplicación iniciada. Presiona 'q' para salir.
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

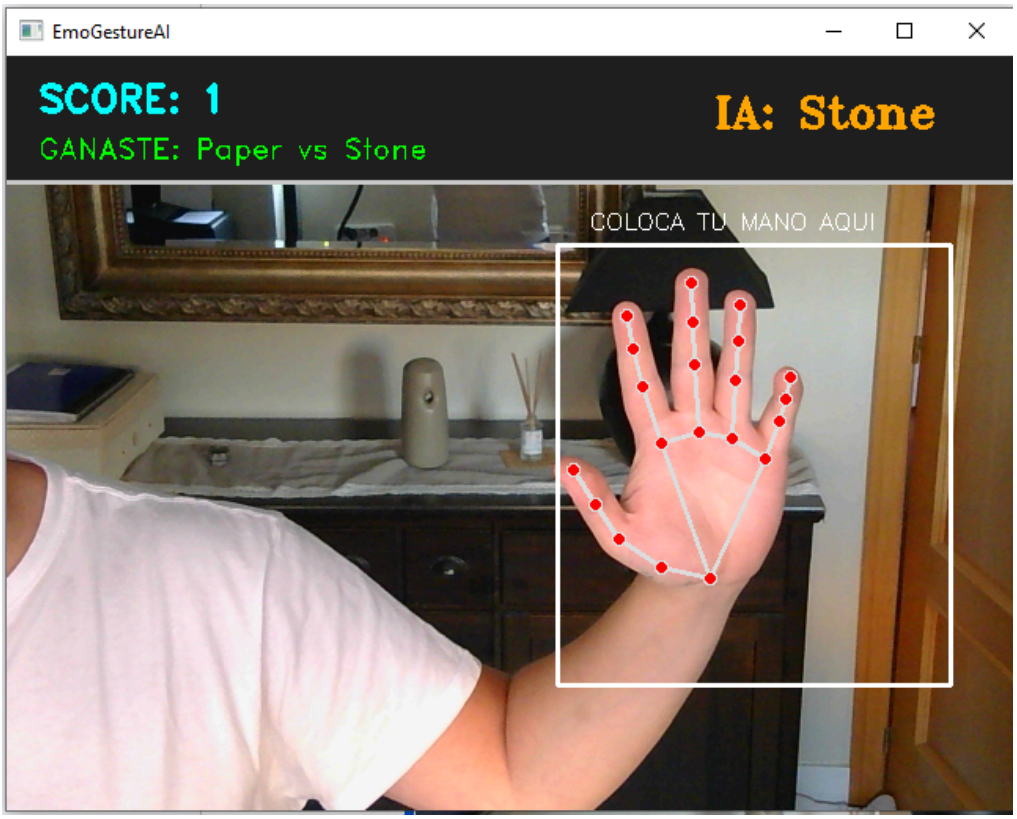
Pantalla inicial antes de comenzar a jugar



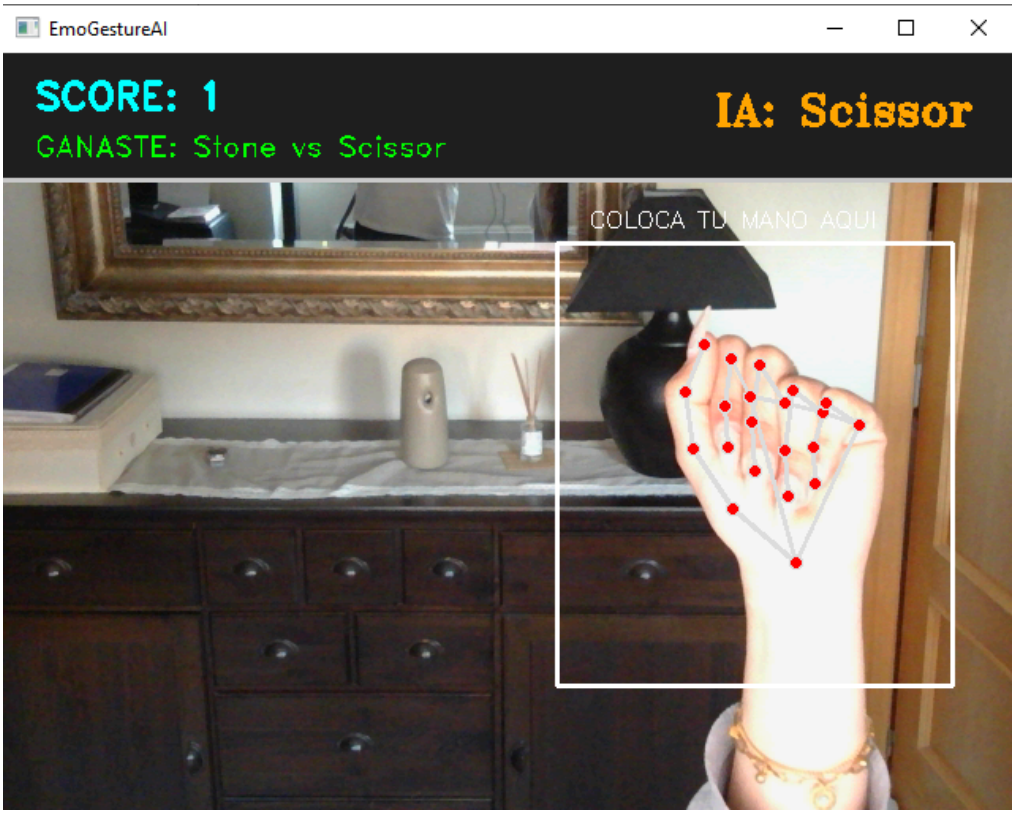
Detecta los puntos de la mano



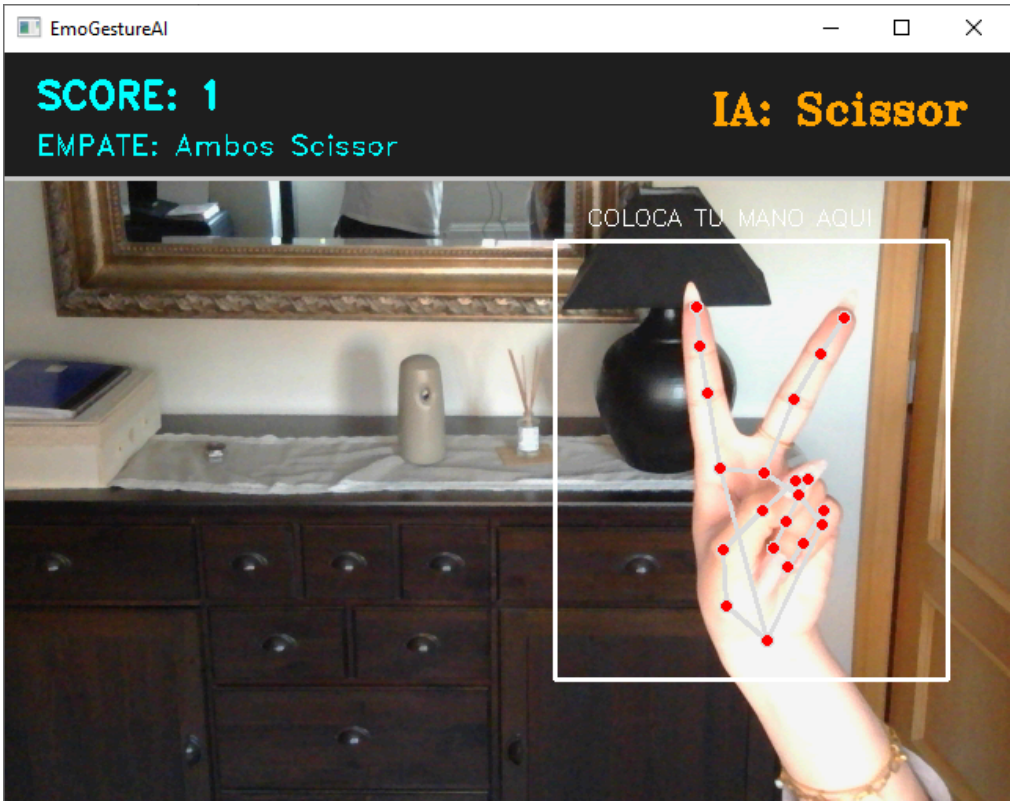
Comienza el juego, el jugador saca papel y la IA Piedra. GANA JUGADOR



Segunda Partida, el jugador saca Piedra y la IA Tijera. GANA JUGADOR



Ambos sacan tijera. EMPATE

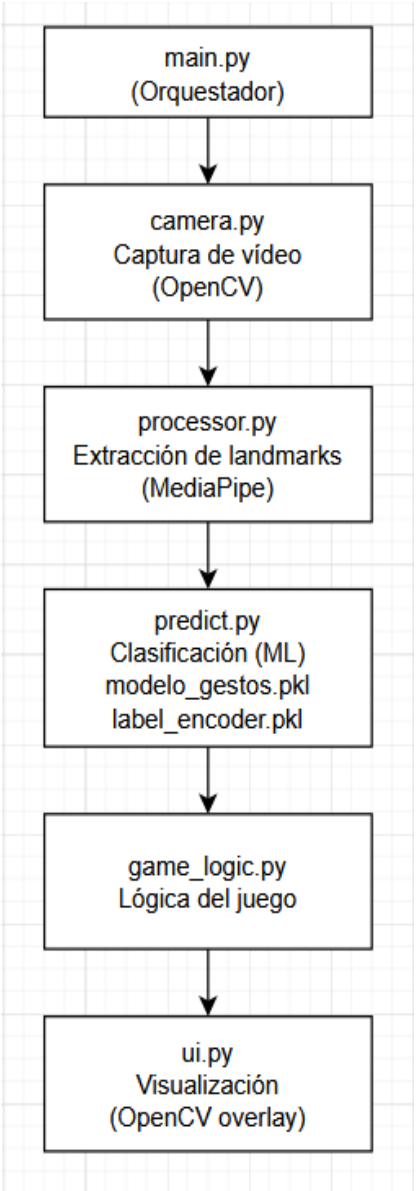


4. Arquitectura real implementada

La arquitectura del sistema sigue un enfoque modular basado en separación de responsabilidades, permitiendo el desarrollo independiente de cada componente y facilitando su mantenimiento, escalabilidad y evolución.

El sistema se divide en dos bloques principales: el pipeline de inferencia en tiempo real y el pipeline de entrenamiento del modelo.

- Pipeline principal:



El flujo principal del sistema está orquestado desde el módulo **main.py**, que actúa como punto de entrada y coordina la ejecución del resto de componentes.

En primer lugar, el módulo **camera.py** se encarga de la captura de vídeo en tiempo real mediante OpenCV, proporcionando los frames que serán procesados por el sistema.

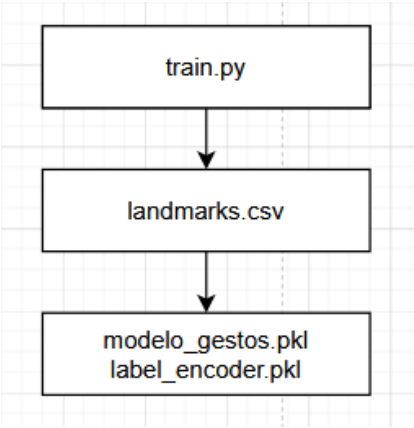
A continuación, el módulo **processor.py** utiliza MediaPipe para detectar la mano del usuario y extraer sus puntos clave (landmarks), generando un vector de características numéricas (hasta 63 valores) que representa la posición de la mano.

Este vector es enviado al módulo **predict.py**, que constituye el núcleo del sistema de Machine Learning. Este módulo carga el modelo previamente entrenado **(modelo_gestos.pkl)** junto con el codificador de etiquetas **(label_encoder.pkl)** y realiza la clasificación del gesto. Además, aplica un umbral de confianza para validar las predicciones y evitar resultados erróneos.

El gesto reconocido es posteriormente procesado por el módulo **game_logic.py**, que implementa la lógica del juego de “Piedra, Papel o Tijera”, generando la jugada de la IA y determinando el resultado de cada ronda.

Finalmente, el módulo **ui.py** se encarga de la visualización en tiempo real, superponiendo sobre el flujo de vídeo información relevante como el gesto detectado, el resultado de la partida y el marcador.

- Pipeline de entrenamiento:



De forma independiente, el módulo **train.py** permite el entrenamiento offline del modelo a partir del dataset de landmarks (**landmarks.csv**). Este proceso genera los artefactos necesarios para la inferencia (**modelo_gestos.pkl** y **label_encoder.pkl**), que son utilizados posteriormente por el sistema en tiempo real.

Esta separación entre entrenamiento e inferencia permite mantener un sistema ligero y optimizado para ejecución en CPU, al tiempo que facilita la mejora continua del modelo sin afectar al resto del sistema.

5. Índice de código (estructura del proyecto)

```
EMOGESTUREAI/
├── data/
│   └── landmarks.csv
├── docs/
│   ├── arquitectura.png
│   ├── Fase0.pdf
│   ├── Fase1.pdf
│   └── services.py
├── environment/
│   └── requirements.txt
├── models/
│   ├── confusion_matrix.png
│   ├── label_encoder.pkl
│   └── modelo_gestos.pkl
├── src/
│   ├── ml
│       ├── predict.py
│       ├── processor.py
│       └── train.py
│   ├── camera.py
│   ├── game_logic.py
│   ├── main.py
│   └── ui.py
```

6. Código completo del proyecto

- **predict.py**
  - Ruta: src/ml/predict.py
  - Código:

```
import os
import joblib
import numpy as np

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PATH_MODELO = os.path.join(BASE_DIR, "..", "..", "models",
"modelo_gestos.pkl")
PATH_ENCODER = os.path.join(
    BASE_DIR, "..", "..", "models", "label_encoder.pkl")

if not os.path.exists(PATH_MODELO):
    raise FileNotFoundError(f"No se encuentra el modelo en
{PATH_MODELO}")

_modelo = joblib.load(PATH_MODELO)
_encoder = joblib.load(PATH_ENCODER)

UMBRAL_CONFIANZA = 0.50
N_FEATURES = _modelo.n_features_in_


def predecir_gesto(landmarks: list) -> dict:
    if len(landmarks) != N_FEATURES:
        return {
            "gesto": f"Error: modelo espera {N_FEATURES}",
            "confianza": 0,
            "reconocido": False
        }

    X = np.array(landmarks).reshape(1, -1)
    probas = _modelo.predict_proba(X)[0]
    confianza = float(np.max(probas))
    idx = int(np.argmax(probas))
    gesto = _encoder.inverse_transform([idx])[0]

    reconocido = confianza >= UMBRAL_CONFIANZA
```

```
return {
    "gesto": gesto if reconocido else None,
    "confianza": confianza,
    "reconocido": reconocido
}
```

- **processor.py**
  - Ruta: src/ml/predict.py
  - Código:

```
import cv2
import mediapipe as mp
from ml.predict import N_FEATURES


class HandProcessor:
    def __init__(self):
        self.mp_hands = mp.solutions.hands
        self.mp_draw = mp.solutions.drawing_utils
        self.hands = self.mp_hands.Hands(
            static_image_mode=False,
            max_num_hands=1,
            min_detection_confidence=0.7,
            min_tracking_confidence=0.5
        )

    def extract_landmarks(self, frame):
        img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = self.hands.process(img_rgb)

        if results.multi_hand_landmarks:
            landmarks_final = []
            hand_landmarks = results.multi_hand_landmarks[0]

            for lm in hand_landmarks.landmark:
                if N_FEATURES == 63:
                    landmarks_final.extend([lm.x, lm.y, lm.z])
                else:
                    landmarks_final.extend([lm.x, lm.y])

            self.mp_draw.draw_landmarks(
                frame, hand_landmarks,
self.mp_hands.HAND_CONNECTIONS)
            return landmarks_final  # Esto devuelve la lista de 42
        return None
```

- **train.py**
  - Ruta: src/ml/train.py
  - Código:

```
import os
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_PATH = os.path.join(BASE_DIR, "..", "..", "data",
"landmarks.csv")
MODELS_DIR = os.path.join(BASE_DIR, "..", "..", "models")


def entrenar_modelo():
    print("Cargando datos...")
    if not os.path.exists(DATA_PATH):
        print(f"Error: No se encuentra {DATA_PATH}")
        return

    df = pd.read_csv(DATA_PATH)
    X = df.drop(columns=["Category"]).values
    y = df["Category"].values

    le = LabelEncoder()
    y_encoded = le.fit_transform(y)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y_encoded, test_size=0.2, random_state=42,
stratify=y_encoded
    )
```

```
    print(f"Entrenando Random Forest con {len(X_train)} muestras...")
    model = RandomForestClassifier(n_estimators=100,
random_state=42)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"Precisión del modelo: {acc*100:.2f}%")
    print(classification_report(y_test, y_pred,
target_names=le.classes_))

    os.makedirs(MODELS_DIR, exist_ok=True)
    joblib.dump(model, os.path.join(MODELS_DIR,
"modelo_gestos.pkl"))
    joblib.dump(le, os.path.join(MODELS_DIR, "label_encoder.pkl"))

    print(f"Modelo guardado en {MODELS_DIR}")


if __name__ == "__main__":
    entrenar_modelo()
```

- **camera.py**
  - Ruta: src/camera.py
  - Código:

```
import cv2


class CameraManager:
    def __init__(self):
        self.cap = None

    def try_connect(self):
        self.cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
        return self.cap.isOpened()

    def get_frame(self):
        if self.cap and self.cap.isOpened():
            ret, frame = self.cap.read()
            if ret:
                return cv2.flip(frame, 1)
        return None

    def release(self):
        if self.cap:
            self.cap.release()
```

- **game_logic.py**
  - Ruta: src/game_logic.py
  - Código:

```
import random


class GameEngine:
    def __init__(self):
        self.moves = ["Paper", "Stone", "Scissor"]
        self.rules = {
            "Stone": "Scissor",
            "Paper": "Stone",
            "Scissor": "Paper"
        }

    def get_ia_move(self):
        return random.choice(self.moves)

    def check_winner(self, user_move, ia_move):
        """Devuelve: 1 (Gana usuario), -1 (Gana IA), 0 (Empate)"""
        if user_move == ia_move:
            return 0
        if self.rules[user_move] == ia_move:
            return 1
        return -1
```

- **main.py**
  - Ruta: src/main.py
  - Código:

```
import cv2
import time

from camera import CameraManager
from game_logic import GameEngine
from ui import UIHandler
from ml.predict import predecir_gesto
from ml.processor import HandProcessor


class EmoGestureApp:
    def __init__(self):
        self.camera = CameraManager()
        if not self.camera.try_connect():
            print("ERROR: No se detecta la cámara. Revisa la conexión.")

        self.ui = UIHandler()
        self.game = GameEngine()
        self.processor = HandProcessor()
        self.score = 0
        self.state = "START"
        self.msg = "Presiona ESPACIO para jugar"
        self.ia_move = None
        self.color_msg = (255, 255, 255)
        self.start_time = 0

    def resolve_game(self, user_gesture):
        self.ia_move = self.game.get_ia_move()
        result = self.game.check_winner(user_gesture, self.ia_move)

        if result == 1:
            self.score += 1
            self.msg = f"GANASTE: {user_gesture} vs {self.ia_move}"
            self.color_msg = (0, 255, 0)
        elif result == -1:
            self.score -= 1
            self.msg = f"PERDISTE: {user_gesture} vs {self.ia_move}"
```

```
            self.color_msg = (0, 0, 255)
        else:
            self.msg = f"EMPATE: Ambos {user_gesture}"
            self.color_msg = (255, 255, 0)

    def run(self):
        print("Aplicación iniciada. Presiona 'q' para salir.")
        while True:
            frame = self.camera.get_frame()

            if frame is None:
                frame = self.ui.draw_error_screen()
            else:
                landmarks = self.processor.extract_landmarks(frame)
                if self.state == "COUNTDOWN":
                    elapsed = time.time() - self.start_time
                    if elapsed < 3:
                        self.msg = f"Lanzando en... {3 - int(elapsed)}"
                    else:
                        if landmarks:
                            res = predecir_gesto(landmarks)
                            if res["reconocido"]:
                                self.resolve_game(res["gesto"])
                            else:
                                self.msg = "Gesto no reconocido"
                        else:
                            self.msg = "No se detecto mano"

                        self.state = "START"

                frame = self.ui.draw_hud(
                    frame, self.msg, self.score, self.ia_move, self.color_msg)

            cv2.imshow("EmoGestureAI", frame)

            key = cv2.waitKey(1) & 0xFF
            if key == ord(' ') and self.state == "START":
                self.state = "COUNTDOWN"
                self.start_time = time.time()
                self.ia_move = None
```

```
            elif key == ord('q'):
                break

        self.camera.release()
        cv2.destroyAllWindows()


if __name__ == "__main__":
    app = EmoGestureApp()
    app.run()
```

- **ui.py**
  - Ruta: src/main.py
  - Código:

```
import cv2
import numpy as np


class UIHandler:
    WHITE = (255, 255, 255)
    GREEN = (0, 255, 0)
    RED = (0, 0, 255)
    BLUE = (255, 100, 0)
    BLACK = (0, 0, 0)

    @staticmethod
    def draw_hud(frame, msg, score, ia_move=None,
result_color=(255, 255, 255)):
        cv2.rectangle(frame, (0, 0), (640, 80), (30, 30, 30), -1)
        cv2.line(frame, (0, 80), (640, 80), (200, 200, 200), 2)

        cv2.putText(frame, f"SCORE: {score}", (20, 35),
                cv2.FONT_HERSHEY_DUPLEX, 0.8, (255, 255, 0), 2)
        cv2.putText(frame, msg, (20, 65),
                cv2.FONT_HERSHEY_DUPLEX, 0.6, result_color, 1)

        if ia_move:
            cv2.putText(frame, f"IA: {ia_move}", (450, 45),
                    cv2.FONT_HERSHEY_TRIPLEX, 0.9, (0, 165, 255), 2)

        cv2.rectangle(frame, (350, 120), (600, 400), (255, 255, 255), 2)
        cv2.putText(frame, "COLOCA TU MANO AQUI", (370, 110),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

        return frame

    @staticmethod
    def draw_error_screen():
        """Crea un frame negro con un aviso de error cuando la cámara
falla."""
        error_frame = np.zeros((480, 640, 3), dtype="uint8")
```

```
txt_main = "ERROR: CAMARA NO DETECTADA"
txt_sub = "Revisa la conexion y reinicia la app"

cv2.putText(error_frame, txt_main, (120, 220),
        cv2.FONT_HERSHEY_DUPLEX, 0.8, (0, 0, 255), 2)
cv2.putText(error_frame, txt_sub, (150, 260),
        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

return error_frame
```

7. Breve explicación por módulo

Machine Learning:

- **processor.py:** Procesamiento sensorial. Utiliza MediaPipe para detectar la mano en el video y extraer las coordenadas (x,y,z)
- **train.py:** Entrenamiento de Inteligencia. El motor que genera el modelo. Toma los datos recolectados y entrena un algoritmo.
- **predict.py**: Inferencia de Resultados. Toma los datos del procesador y los cruza con el modelo entrenado para devolver una etiqueta.

Backend:

- **camera.py:** Gestión de Hardware. Abstrae la complejidad de la webcam. Controla la apertura del dispositivo, resolución y protocolos de video
- **game_logic.py:** Motor de reglas. El cerebro del juego.

Frontend y Orquestación

- **main.py:** Punto de entrada (Orquestador). El archivo principal que une todas las piezas.
- **ui.py:** Interfaz Gráfica. Se encarga de la capa visual sobre OpenCV.

Capa de Datos y Modelos:

- **models:** Almacena los archivos binarios (.pkl) del modelo entrenado y del codificador de etiquetas.
- **data:** Contiene los datasets en CSV y, en la Fase 3, el archivo SQLite para el histórico de partidas.

8. Limitaciones actuales

- Bugs
  - **Latencia en primer frame:** Al cargar el modelo de MediaPipe por primera vez, puede haber congelamiento de 1-2 segundos.
  -
- Atajos
  - **Umbral de Confianza Estático:** El UMBRAL_CONFIANZA está en el código (0.50). Lo ideal sería que fuera un parámetro ajustable desde la UI.
  - **Hardcoded Classes:** Los nombres de los gestos (Stone, Paper, Scissor) están "atados" al modelo. Si se cambian los nombres en el entrenamiento, el GameEngine se rompe.

- Partes incompletas
  - **Persistencia de Datos:** El marcador (score) reside en la memoria volátil; se pierde al cerrar la aplicación.